

```
import os

import json

from tkinter import *

from tkinter import messagebox

from datetime import datetime, timedelta

import pandas as pd

from PIL import Image, ImageTk

import matplotlib.pyplot as plt

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def main_window(username):

    BOOKINGS_FILE = "bookings.json"

    USER_BOOKINGS_FILE = "user_bookings.json"

    PROFITS_FILE = "profits.json"


    def ensure_files():

        for file in [BOOKINGS_FILE, USER_BOOKINGS_FILE, PROFITS_FILE]:

            if not os.path.exists(file):

                with open(file, "w") as f:

                    json.dump({}, f)


    def load_json(path):

        try:

            with open (path, "r") as f:

                data = json.load(f)

                if not isinstance(data, dict):

                    return {}
```

```
        return data
    except:
        return {}
```

```
ensure_files()
```

```
booked_data = load_json(BOOKINGS_FILE)
```

```
user_bookings = load_json(USER_BOOKINGS_FILE)
```

```
profits = load_json(PROFITS_FILE)
```

```
tk = Tk()
```

```
tk.geometry("1200x700")
```

```
tk.configure(background="white")
```

```
tk.title(f"FILMFORG - Welcome {username}")
```

```
Label(tk, text="🎬 FILMFORG", font=("arial", 30, "bold"), bg="black", fg="white",
relief='solid').pack(pady=4)
```

```
movies = [
```

```
    {"name": "BAHUBALI THE EPIC", "image": "bahubali.jpeg"},
```

```
    {"name": "RAJASAB", "image": "rajasaab.jpeg"},
```

```
    {"name": "AA22XA6", "image": "aa22xa6.jpg"},
```

```
]
```

```
theaters = [
```

```
    {"name": "SUPER ACIAN"},
```

```
    {"name": "ART CINEMA"},
```

```
    {"name": "NVN CINEMAS"},
```

```
    {"name": "ASHWINI CINEMAS"},
```

```

{"name": "AAA"},

{"name": "SVCN CINEMAS"},

]

timings = ["11:15 AM", "2:10 PM", "6:15 PM", "9:10 PM"]

dates = [(datetime.now() + timedelta(days=i)).strftime("%Y-%m-%d") for i in
range(5)]

TICKET_PRICE = 150

for movie in movies:

    if not isinstance(booked_data.get(movie["name"]), dict):

        booked_data[movie["name"]] = {}

    for th in theaters:

        if not isinstance(booked_data[movie["name"]].get(th["name"]), dict):

            booked_data[movie["name"]][th["name"]] = {}

        for date in dates:

            if not isinstance(booked_data[movie["name"]][th["name"]].get(date), dict):

                booked_data[movie["name"]][th["name"]][date] = {}

            for time in timings:

                if not isinstance(booked_data[movie["name"]][th["name"]][date].get(time),
dict):

                    booked_data[movie["name"]][th["name"]][date][time] = {"seats": []}

def save_all():

    with open(BOOKINGS_FILE, "w") as f: json.dump(booked_data, f, indent=4)

    with open(USER_BOOKINGS_FILE, "w") as f: json.dump(user_bookings, f,
indent=4)

    with open(PROFITS_FILE, "w") as f: json.dump(profits, f, indent=4)

```

```

imgs = []

right_frame = Frame(tk, bg="#f7f7f7", width=400, height=400, relief="solid", bd=1)
right_frame.place(x=750, y=100)

down_frame = Frame(tk, bg="#f7f7f7", width=600, height=600)
down_frame.place(x=60, y=300)

Label(tk, text=f"Welcome, {username}", bg="black", fg="white", font=("Arial", 12,
"bold")).place(x=10, y=10)

def show_user_bookings():
    user_history = user_bookings.get(username, [])

    history = Toplevel(tk)
    history.title(f"{username}'s Bookings")
    history.geometry("500x400")

    Label(history, text=f"{username}'s Booking History", font=("Arial", 16,
"bold")).pack(pady=10)

    if not user_history:
        Label(history, text="No previous bookings!").pack()
    else:
        for b in user_history:
            Frame(history, height=2, bd=1, relief="sunken").pack(fill=X, padx=10,
pady=5)

            Label(history, text=f"🎬 {b['movie']} - {b['theater']}").pack()
            Label(history, text=f"📅 {b['date']} | ⌚ {b['time']}").pack()
            Label(history, text=f"📍 Seats: {', '.join(b['seats'])} | ₹{b['amount']}").pack()

```

```
Button(tk, text="My Bookings", bg="orange", fg="white", font=("Arial", 12, "bold"),
command=show_user_bookings).place(x=1000, y=10)
```

```
def show_bookings(time, theater, movie, date):

    for w in right_frame.winfo_children(): w.destroy()

    Label(right_frame, text=f"{movie['name']} ({date})", font=("Arial", 16, "bold"),
bg="#f7f7f7").pack(pady=10)

    Label(right_frame, text=theater['name'], font=("Arial", 14, "bold"),
bg="#f7f7f7").pack(pady=5)

    Label(right_frame, text=f"Show: {time}", font=("Arial", 12),
bg="#f7f7f7").pack(pady=5)

    Label(right_frame, text=f"Rs {TICKET_PRICE}", font=("Arial", 12, "bold"),
bg="#f7f7f7").pack(pady=5)
```

```
booked = booked_data[movie["name"]][theater["name"]][date][time]["seats"]
```

```
seat_frame = Frame(right_frame, bg="#f7f7f7")
```

```
seat_frame.pack(pady=10)
```

```
selected = []
```

```
rows = [chr(i) for i in range(65, 75)]
```

```
for r in rows:
```

```
    for c in range(1, 11):
```

```
        seat = f"{r}{c}"
```

```
        color = "lightgreen"
```

```
        if seat in booked:
```

```
            color, state = "red", DISABLED
```

```
        else:
```

```
            state = NORMAL
```

```
        btn = Button(seat_frame, text=seat, width=4, bg=color, state=state)
```

```
btn.config(command=lambda b=btn, s=seat: toggle(b, s))
```

```
btn.grid(row=rows.index(r), column=c, padx=3, pady=3)
```

```
def toggle(btn, seat):
```

```
    if seat in selected:
```

```
        selected.remove(seat)
```

```
        btn.config(bg="lightgreen")
```

```
    else:
```

```
        selected.append(seat)
```

```
        btn.config(bg="yellow")
```

```
def payment_screen():
```

```
    if not selected:
```

```
        messagebox.showwarning("No Seats", "Please select seats!")
```

```
        return
```

```
    for w in right_frame.winfo_children(): w.destroy()
```

```
    pay_frame = Frame(right_frame, bg="#f7f7f7")
```

```
    pay_frame.pack(pady=20)
```

```
    Label(pay_frame, text="Select Payment Method:", font=("Arial", 12, "bold"),  
bg="#f7f7f7").pack()
```

```
    method = StringVar(value="QR")
```

```
    for m in ["QR", "Card", "Cash"]:
```

```
        Radiobutton(pay_frame, text=m, variable=method, value=m,  
bg="#f7f7f7").pack(anchor="w")
```

```
def proceed_payment():
```

```
    if method.get() == "QR":
```

```

        show_qr(movie, theater, date, time, selected)

    else:

        finalize_booking(movie, theater, date, time, selected,
len(selected)*TICKET_PRICE)

        Button(pay_frame, text="Proceed to Pay", bg="blue", fg="white",
command=proceed_payment).pack(pady=10)


        Button(right_frame, text="Confirm Booking", bg="green", fg="white",
command=payment_screen).pack(pady=10)


def show_qr(movie, theater, date, time, seats):

    for w in right_frame.winfo_children(): w.destroy()

    Label(right_frame, text="Scan QR to Pay", font=("Arial", 14, "bold"),
bg="#f7f7f7").pack(pady=10)

    amount = len(seats)*TICKET_PRICE

    upi_link =
f"upi://pay?pa=6303672524@upi&pn=FILMFORG&am={amount}&cu=INR"

    qr = qrcode.make(upi_link)

    qr_path = "qr.png"

    qr.save(qr_path)

    img = Image.open(qr_path).resize((180, 180))

    img_tk = ImageTk.PhotoImage(img)

    Label(right_frame, image=img_tk, bg="#f7f7f7").pack()

    right_frame.image = img_tk

    Label(right_frame, text=f"Amount: ₹{amount}", bg="#f7f7f7").pack(pady=10)

    Button(right_frame, text="Payment Done 

```

```
command=lambda: finalize_booking(movie, theater, date, time, seats,
amount)).pack(pady=10)
```

```
def finalize_booking(movie, theater, date, time, seats, amount):
    for w in right_frame.winfo_children():w.destroy()
    booked_data[movie["name"]][theater["name"]][date][time]["seats"].extend(seats)
    if username not in user_bookings: user_bookings[username] = []
    user_bookings[username].append({
        "movie": movie["name"], "theater": theater["name"],
        "date": date, "time": time, "seats": seats, "amount": amount
    })
    today = datetime.now().strftime("%Y-%m-%d")
    profits[today] = profits.get(today, 0) + amount
    save_all()
    messagebox.showinfo("Success ☒ ", f"Booking Confirmed!\nAmount
₹{amount}")
    show_ticket(movie, theater, date, time, seats, amount)
```

```
def show_ticket(movie, theater, date, time, seats, amount):
    t = Toplevel(tk)
    t.title("🎫 Ticket")
    Label(t, text="🎬 FILMFORG RECEIPT", font=("Arial", 14, "bold")).pack(pady=10)
    Label(t, text=f"Movie: {movie['name']}").pack()
    Label(t, text=f"Theater: {theater['name']}").pack()
    Label(t, text=f>Date: {date} | Time: {time}").pack()
    Label(t, text=f"Seats: {', '.join(seats)}").pack()
    Label(t, text=f"Amount Paid: ₹{amount}").pack()
```



```

Label(t, text="☒ Booking Confirmed").pack(pady=10)

Button(t, text="Close", command=t.destroy).pack()

def show_theaters(movie):

    for w in right_frame.winfo_children(): w.destroy()

    for w in down_frame.winfo_children(): w.destroy()

    Label(down_frame, text=f"{movie['name']}", font=("Arial", 18, "bold"),
bg="white").pack(pady=10)

    Label(down_frame, text="Select Date:", font=("Arial", 12, "bold"),
bg="white").pack()

    for d in dates:

        Button(down_frame, text=d, bg="lightblue", command=lambda dd=d:
show_times(movie, dd)).pack(pady=2)

def show_times(movie, date):

    today = datetime.now().strftime("%Y-%m-%d")

    for w in down_frame.winfo_children(): w.destroy()

    Label(down_frame, text=f"{movie['name']} - {date}", font=("Arial", 16,
"bold")).pack(pady=10)

    for th in theaters:

        Label(down_frame, text=th["name"], font=("Arial", 14, "bold")).pack()

        f = Frame(down_frame)

        f.pack()

        for t in timings:

            if(time>t and date==today):

                Button(f, text=t, bg="RED",state=DISABLED).pack(side=LEFT,padx=4)

```

```

        else:

            Button(f, text=t, bg="lightgreen",

                   command=lambda t=t, th=th, m=movie, d=date: show_bookings(t, th,
m, d)).pack(side=LEFT, padx=4)

x, y = 50, 100
for m in movies:

    img = Image.open(m["image"]).resize((150, 150))

    img_tk = ImageTk.PhotoImage(img)

    imgs.append(img_tk)

    Button(tk, image=img_tk, command=lambda mm=m:
show_theaters(mm)).place(x=x, y=y)

    x += 200

tk.mainloop()

def analysis():

    def load_data():

        try:

            with open("bookings.json", "r") as f:

                data = json.load(f)

        except (FileNotFoundError, json.JSONDecodeError):

            messagebox.showerror("Error", "bookings.json not found or invalid!")

            return None

    rows = []

    for movie, theaters in data.items():

```

```
for theater, dates in theaters.items():
    for date, showtimes in dates.items():
        for showtime, details in showtimes.items():
            seats = details.get("seats", [])
            if seats:
                rows.append({
                    "Movie": movie,
                    "Theater": theater,
                    "Date": date,
                    "Showtime": showtime,
                    "Seats_Booked": len(seats)
                })
```

```
if not rows:
    messagebox.showinfo("Info", "No booked seats found.")
    return None
```

```
return pd.DataFrame(rows)
```

```
def create_charts(df, container):
    for widget in container.winfo_children():
        widget.destroy()
```

```
fig, axes = plt.subplots(2, 2, figsize=(11, 8), dpi=120)
plt.subplots_adjust(hspace=0.5, wspace=0.3)
```

```

movie_data = df.groupby("Movie")["Seats_Booked"].sum()
movie_data.plot(kind="bar", ax=axes[0, 0], color="#4FC3F7")
axes[0, 0].set_title("🎬 Seats Booked per Movie", fontsize=11, pad=10)
axes[0, 0].set_ylabel("Total Seats", fontsize=9)
axes[0, 0].set_xticklabels(movie_data.index, rotation=15, ha="right")

theater_data = df.groupby("Theater")["Seats_Booked"].sum()
theater_data.plot(kind="bar", ax=axes[0, 1], color="#81C784")
axes[0, 1].set_title("🏛️ Seats Booked per Theater", fontsize=11, pad=10)
axes[0, 1].set_ylabel("Total Seats", fontsize=9)
axes[0, 1].set_xticklabels(theater_data.index, rotation=15, ha="right")

date_data = df.groupby("Date")["Seats_Booked"].sum()
date_data.plot(kind="line", marker="o", ax=axes[1, 0], color="#FFB74D")
axes[1, 0].set_title("📅 Seats Booked Over Time", fontsize=11, pad=10)
axes[1, 0].set_ylabel("Seats", fontsize=9)
axes[1, 0].tick_params(axis="x", rotation=15)

pivot = df.pivot_table(index="Theater", columns="Movie",
values="Seats_Booked", aggfunc="sum", fill_value=0)

im = axes[1, 1].imshow(pivot, cmap="coolwarm", aspect="auto")
axes[1, 1].set_title("🔥 Heatmap: Movies vs Theaters", fontsize=11, pad=10)
axes[1, 1].set_xticks(range(len(pivot.columns)))
axes[1, 1].set_xticklabels(pivot.columns, rotation=30, ha="right", fontsize=8)
axes[1, 1].set_yticks(range(len(pivot.index)))
axes[1, 1].set_yticklabels(pivot.index, fontsize=8)
fig.colorbar(im, ax=axes[1, 1], orientation="vertical", fraction=0.046, pad=0.04)

```

```

canvas = FigureCanvasTkAgg(fig, master=container)

canvas.draw()

canvas.get_tk_widget().pack(fill=BOTH, expand=True)

def main():

    root = Tk()

    root.title("📊 Booking Analytics Dashboard")

    root.geometry("1232x768")

    root.config(bg="white")

    title = Label(root, text="📊 Booking Data Dashboard", font=("Arial", 20, "bold"),
bg="white", fg="#111")

    title.pack(pady=10)

    frame = Frame(root, bg="white")

    frame.pack(fill=BOTH, expand=True)

    def refresh():

        df = load_data()

        if df is not None:

            create_charts(df, frame)

    Button(root, text="🔄 Load & Refresh Charts", font=("Arial", 12, "bold"),
bg="#4CAF50", fg="white", padx=15, pady=5,
command=refresh).pack(pady=10)

```

```
root.mainloop()
```

```
if __name__ == "__main__":
```

```
    main()
```

```
def signup_menu(spassword, suser_name, phone_entry, s1password,  
sign_up_window):
```

```
    name = suser_name.get().strip()
```

```
    password = spassword.get().strip()
```

```
    cpassword = s1password.get().strip()
```

```
    phone_data = phone_entry.get().strip()
```

```
    if not name or not password or not cpassword or not phone_data:
```

```
        messagebox.showerror("Error", "Please fill all fields")
```

```
        return
```

```
    if password != cpassword:
```

```
        messagebox.showerror("Error", "Passwords do not match")
```

```
        return
```

```
filename = "user_details.json"
```

```
if os.path.exists(filename) and os.path.getsize(filename) > 0:
```

```
    with open(filename, "r") as f:
```

```
        try:
```

```
            details = json.load(f)
```

```
            if not all(isinstance(d, dict) for d in details):
```

```
                details = []
```

```
        except json.JSONDecodeError:
```

```

        details = []
else:
    details = []

for detail in details:
    if detail.get("name") == name:
        messagebox.showerror("Error", "Username already exists")
        return

details.append({"name": name, "password": password, "phone": phone_data})

with open(filename, "w") as f:
    json.dump(details, f, indent=4)

messagebox.showinfo("Success", "User created successfully")
sign_up_window.destroy()
login_section()

def sign_up():
    sign_up_window = Tk()
    sign_up_window.title("Sign Up")
    sign_up_window.geometry("500x400")
    sign_up_window.configure(bg="white")

    Label(sign_up_window, text="SIGNUP", fg="white", bg="black", font=("Arial", 20,
"bold")).place(x=200, y=10)

```

```
Label(sign_up_window, text="USER NAME:-", fg="white", bg="black", font=("Arial", 15, "bold")).place(x=10, y=90)
```

```
suser_name = Entry(sign_up_window, width=15, font=("Arial", 20,))
```

```
suser_name.place(x=180, y=90)
```

```
Label(sign_up_window, text="PASSWORD:-", fg="white", bg="black", font=("Arial", 15, "bold")).place(x=10, y=130)
```

```
spassword = Entry(sign_up_window, width=15, font=("Arial", 20,), show="*")
```

```
spassword.place(x=180, y=130)
```

```
Label(sign_up_window, text="CONFIRM PASSWORD:-", fg="white", bg="black", font=("Arial", 15, "bold")).place(x=10, y=170)
```

```
s1password = Entry(sign_up_window, width=15, font=("Arial", 20,), show="*")
```

```
s1password.place(x=180, y=170)
```

```
Label(sign_up_window, text="MOBILE NO:-", fg="white", bg="black", font=("Arial", 15, "bold")).place(x=10, y=210)
```

```
phone_entry = Entry(sign_up_window, width=15, font=("Arial", 20,))
```

```
phone_entry.place(x=180, y=210)
```

```
Button(sign_up_window, text="SIGNUP", fg="blue", bg="white", font=("Arial", 16, "bold"),
```

```
command=lambda: signup_menu(spassword, suser_name, phone_entry, s1password, sign_up_window)).place(x=190, y=270)
```

```
Button(sign_up_window, text="Have Account?", fg="blue", bg="white", command=lambda: close_this()).place(x=195, y=320)
```



```
def close_this():  
    sign_up_window.destroy()  
    login_section()  
  
sign_up_window.resizable(False, False)  
sign_up_window.mainloop()
```

```
def login_section():  
    login_window = Tk()  
    login_window.title("Login System")  
    login_window.geometry("400x400")  
    login_window.config(bg="white")
```

```
def login_check():  
    name = user_name.get().strip()  
    pwd = password.get().strip()  
  
    if not os.path.exists("user_details.json"):  
        messagebox.showerror("Error", "No users registered yet!")  
        return  
  
    with open("user_details.json", "r") as f:  
        try:  
            users = json.load(f)  
        except json.JSONDecodeError:
```

```

users = []

found = False

for user in users:
    if isinstance(user, dict) and user.get("name") == name and
user.get("password") == pwd:
        found = True
        messagebox.showinfo("Login Successful", f"Welcome {name}!")
        login_window.destroy()
        main_window(name)
        break

if not found:
    messagebox.showerror("Error", "Invalid username or password!")

def closing():
    login_window.destroy()
    sign_up()

Label(login_window, text="LOGIN", fg="white", bg="black", font=("Arial", 20,
"bold")).place(x=150, y=10)

Label(login_window, text="USER NAME:-", fg="white", bg="black", font=("Arial",
15, "bold")).place(x=10, y=90)

user_name = Entry(login_window, width=15, font=("Arial", 20,))

user_name.place(x=160, y=90)

```

```
Label(login_window, text="PASSWORD:-", fg="white", bg="black", font=("Arial", 15, "bold")).place(x=10, y=140)
```

```
password = Entry(login_window, width=15, font=("Arial", 20,), show="*")
```

```
password.place(x=160, y=140)
```

```
Button(login_window, text="LOGIN", fg="blue", bg="white", font=("Arial", 16, "bold"), command=lambda: login_check()).place(x=160, y=200)
```

```
Button(login_window, text="Don't have account?", fg="blue", bg="white", command=lambda: closing()).place(x=150, y=250)
```

```
Button(login_window, text="show analysis", fg="blue", bg="white", command=lambda: analysis()).place(x=160, y=280)
```

```
login_window.resizable(False, False)
```

```
login_window.mainloop()
```

```
login_section()
```